

# Computer Architectures

## Exam of 24.02.2023 – part II

### Question 1

Write the `KaprekarRoutine` subroutine, which receives in input a 32-bit unsigned number  $a$ , ranging between 1000 and 9999. The subroutine implements the following tasks:

- it computes a new number  $b$  by sorting the digits of  $a$  by descending order
- it computes a new number  $c$  by sorting the digits of  $a$  by ascending order
- it returns  $d = b - c$

Example.

$a = 3075$

The subroutine computes  $b = 7530$  and  $c = 0357$ .

The returning value is  $d = 7530 - 357 = 7173$ .

You can obtain the digits of a number (from the least significant to the most significant) by repeatedly dividing by 10 and taking the remainder. For example:

$3075 / 10 = 307$  with remainder 5

$307 / 10 = 30$  with remainder 7

$30 / 10 = 3$  with remainder 0

$3 / 10 = 0$  with remainder 3

After sorting the digits, you can build the new numbers  $b$  and  $c$  by repeated multiplication by 10. In the previous example, the digits in descending order are 7, 5, 3, 0. You can build  $b$  as follows:

- first iteration:  $b = 7$
- second iteration:  $b = 7 * 10 + 5 = 75$
- third iteration:  $b = 75 * 10 + 3 = 753$
- fourth iteration:  $b = 753 * 10 + 0 = 7530$

# Computer Architectures

## Exam of 24.02.2023 – part II

### Question 2

Implement the handler for the supervisor call 50.

The handler repeatedly calls the `KaprekarRoutine` subroutine developed in the previous exercise, passing the value stored in `r0`. The loop ends when the `KaprekarRoutine` subroutine returns 6174. The handler sets `r6` equal to the number of iterations (i.e., the number of times the subroutine was called).

#### Examples

If the supervisor call is called with `r0 = 3075`, the subsequent values of `r0` are: 7173 -> 6354 -> 3087 -> 8352 -> 6174 and the handler returns 5.

It is proved that, for any four-digit numbers in base 10 (with at least two distinct digits), the Kaprekar's routine always yields the number 6174 after at most seven iterations. The value 6174 is known as Kaprekar's constant.

Note: within the handler, you can get the 16-bit operating code of the supervisor call by accessing the stack at position `SP + 24` (you can assume that the calling program was using the main stack). The immediate value of the supervisor call is stored in the least significant byte.